In the following snippet, the worker is created right before the `ask`. Then the `ask` uses the worker reference and `Worker.Parse` to send the request. Finally, the callback processes the responses for success and failure. It creates one report or another, depending on whether the response is a success or a failure:

```
object Manager {
    ...

    val worker: ActorRef[Worker.Command] =                  ❶
         context.spawn(Worker(file), s"worker-$text")       ❶
    context.ask(worker, Worker.Parse) {                     ❷
      case Success(Worker.Done) =>                          ❸
        Report(s"$text parsed by ${worker.path.name}")      ❸
      case Failure(ex) =>                                   ❸
        Report(s"parsing '$text has failed with [${ex.getMessage()}")

    ...
}
```

❶ Creates a worker

❷ Sends the message to the worker

❸ Defines the callback over the two possible outcomes

This snippet requires a timeout that specifies how long the manager will wait for the response. This is set with `timeout`:

```
implicit val timeout: Timeout = Timeout(3, SECONDS)
```

This implicit value is passed to the `ask` by Scala because `ask` has an implicit argument of this type, as you can see in this simplified version of its signature:

```
package akka.actor.typed.scaladsl

trait ActorContext ... {
```